

Through this interaction, I verified that Gemini Pro not only produces syntactically correct code but also supports conceptual understanding by linking equations to implementation. After executing the code, the gradient ascent trajectory behaved as expected—it converged toward local maxima and visualized smooth parameter updates.

You are a code assistant. Please finish the TODO part and follow the structure already given.

Hello! I see you're working on implementing a **gradient ascent** step. Here is the completed `gd_step` function based on the provided mathematical formula and the existing structure.

```
Python

def gd_step(x, y, eta):
    # Calculate the gradients of f with respect to x and y
    dx, dy = grad_f(x, y)

    # Implement the gradient ascent update rule:
    # x_{t+1} = x_t + \eta * (\partial f / \partial x)(x_t, y_t)
    # y_{t+1} = y_t + \eta * (\partial f / \partial y)(x_t, y_t)

    new_x = x + eta * dx
    new_y = y + eta * dy

    # Return the new parameters as a tuple
    return new_x, new_y
```

Key Concepts

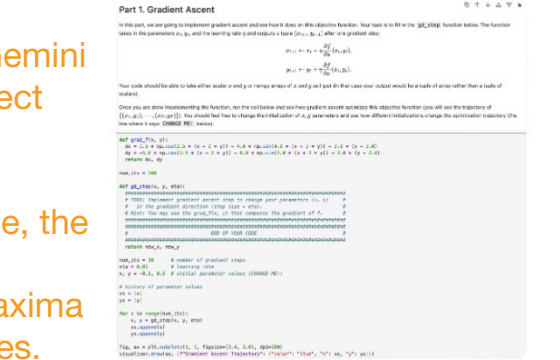
- Gradient Ascent:** The goal is to **maximize** an objective function $f(x, y)$. This is done by iteratively taking steps proportional to the **positive** gradient of the function at the current point.
- Update Rule:** The provided update rules are:

$$x_{t+1} \leftarrow x_t + \eta \frac{\partial f}{\partial x}(x_t, y_t)$$
$$y_{t+1} \leftarrow y_t + \eta \frac{\partial f}{\partial y}(x_t, y_t)$$

where η is the **learning rate** (step size).

The implemented code uses `grad_f(x, y)` to get the partial derivatives `dx` and `dy`, and then applies this formula to calculate `new_x` and `new_y`.

Would you like me to explain how the provided `grad_f(x, y)` function calculates the derivatives or help with interpreting the visualization?



Part 1.Gradient Ascent

In this part, we are going to implement gradient ascent and see how it does on this objective function. Your task is to fill in the `gd_step` function below. The function takes in the parameters x, y , and the learning rate η and outputs a tuple (x_{t+1}, y_{t+1}) after one gradient step:

$$x_{t+1} \leftarrow x_t + \eta \frac{\partial f}{\partial x}(x_t, y_t),$$
$$y_{t+1} \leftarrow y_t + \eta \frac{\partial f}{\partial y}(x_t, y_t).$$

Your code should be able to take either scalar x and y or numpy arrays of x and y as input (in that case your output would be a tuple of array rather than a tuple of scalars).

Once you are done implementing the function, run the cell below and see how gradient ascent optimizes this objective function (you will see the trajectory of (x_t, y_t) for $t = 0, 1, \dots, (num_its - 1)$). You should feel free to change the initialization of x, y parameters and see how different initializations change the optimization trajectory line where it says **CHANGE ME!** below).

```
def grad_f(x, y):
    dx = 2.5 + np.cos(2.5 * (x - 2 * y)) - 4.0 * np.sin(4.0 * (x + 2 * y)) - 2.0 * (x - 2.0)
    dy = -5.0 + np.cos(2.5 * (x - 2 * y)) - 8.0 * np.sin(4.0 * (x + 2 * y)) - 2.0 * (y - 2.0)
    return dx, dy

num_its = 100

def gd_step(x, y, eta):
    # TODO: Implement gradient ascent step to change your parameters (x, y)
    # in the gradient direction (step size = eta).
    # Hint: You may use the grad_f(x, y) that computes the gradient of f.
    # END OF YOUR CODE

    # Calculate the gradients of f with respect to x and y
    dx, dy = grad_f(x, y)

    # Implement the gradient ascent update rule:
    # x_{t+1} = x_t + \eta * (\partial f / \partial x)(x_t, y_t)
    # y_{t+1} = y_t + \eta * (\partial f / \partial y)(x_t, y_t)

    new_x = x + eta * dx
    new_y = y + eta * dy

    # Return the new parameters as a tuple
    return new_x, new_y

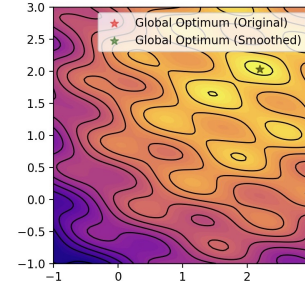
num_its = 30
eta = 0.01
x, y = -0.3, 0.5 # Initial parameter values (CHANGE ME!)

# History of parameter values
xs = []
ys = []

for i in range(num_its):
    x, y = gd_step(x, y, eta)
    xs.append(x)
    ys.append(y)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3.0), dpi=200)
visualizer.draw(ax, ("Gradient Ascent Trajectory: ("color": "blue", "x": xs, "y": ys))
```





You're now moving on to **Part 3a: Finite-difference method** to approximate the gradient of the smoothed function $\tilde{f}_\sigma(x, y)$.

The finite-difference approximation for the partial derivatives is given by:

$$\frac{\partial \tilde{f}_\sigma}{\partial x}(x, y) \approx \frac{\tilde{f}_\sigma(x + \delta, y) - \tilde{f}_\sigma(x, y)}{\delta}$$

$$\frac{\partial \tilde{f}_\sigma}{\partial y}(x, y) \approx \frac{\tilde{f}_\sigma(x, y + \delta) - \tilde{f}_\sigma(x, y)}{\delta}$$

You need to implement the `finite_diff_grad_step` function, which uses these approximations to perform a gradient ascent step.

Here is the completed `finite_diff_grad_step` function.

```
Python

import numpy as np

# Note: This implementation assumes the 'smoothed_f' function
# and the original objective function 'f' are available in the scope.
# 'smoothed_f' from the previous part:
def smoothed_f(f, x, y, sigma, N):
    x_samples = np.random.normal(loc=x, scale=sigma, size=N)
    y_samples = np.random.normal(loc=y, scale=sigma, size=N)
    return np.mean(f(x_samples, y_samples))

def finite_diff_grad_step(f, x, y, delta, N, eta, sigma):
    # dx = np.random.normal(x, sigma, size=N) # Tilde x samples (already provide)
    # dy = np.random.normal(y, sigma, size=N) # Tilde y samples (already provide)
    # =====
    # TODO: Implement the finite-difference gradient of the smoothed #
    # function f. #
    # =====

    # 1. Compute the smoothed function estimate at the current point (x, y)
    f_current = smoothed_f(f, x, y, sigma, N)

    # 2. Compute the smoothed function estimate at the point (x + delta, y)
    f_x_plus_delta = smoothed_f(f, x + delta, y, sigma, N)

    # 3. Compute the smoothed function estimate at the point (x, y + delta)
    f_y_plus_delta = smoothed_f(f, x, y + delta, sigma, N)

    # 4. Approximate the partial derivatives (gradients) using finite differences

    # Gradient w.r.t x (dx_hat)
    dx_hat = (f_x_plus_delta - f_current) / delta

    # Gradient w.r.t y (dy_hat)
    dy_hat = (f_y_plus_delta - f_current) / delta

    # 5. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y
# =====
# END OF YOUR CODE
# =====
```

provide more argument then requirements

smoothed_f() takes 4 positional arguments but 5 were given

This error message: `smoothed_f() takes 4 positional arguments but 5 were given` means you are passing **one too many arguments** when calling the `smoothed_f` function.

Tracing the Error

Based on the code structure from **Part 2** and **Part 3a** you provided, the function signature for `smoothed_f` is defined as:

```
Python

def smoothed_f(f, x, y, sigma, N):
```

This function definition takes **5 positional arguments**: `f`, `x`, `y`, `sigma`, and `N`.

The typical error occurs when this function is called inside another function, like the `finite_diff_grad_step` function, and an extra, unnecessary argument is included in the call.

How to Fix It

You need to check how `smoothed_f` is being called within your `finite_diff_grad_step` function (or wherever the error is occurring) and ensure you are only passing the correct arguments.

For example, if you are using a slightly modified version of `smoothed_f` that omits the original function `f` (or `N`), you must adjust the call.

If your function was redefined to only take `x`, `y`, `sigma`, and `N` (4 arguments):

```
Python

# If redefined (4 arguments):
def smoothed_f(x, y, sigma, N):
    ...
```

Then your call would need to look like this:

```
Python

# Call for a 4-argument function:
f_current = smoothed_f(x, y, sigma, N)
```

If your `smoothed_f` definition truly has 5 arguments (as shown above), the error is likely that you are passing 6 arguments in the function call.

Actionable Step: Review all places where `smoothed_f(...)` is called in your code and verify that it is only passed five values, corresponding to `f`, `x`, `y`, `sigma`, and `N`.

Explanation

- Estimating $\tilde{f}_\sigma$:** Since we cannot compute $\tilde{f}_\sigma(x, y)$ exactly, we use the `smoothed_f` function (from your Part 2) with `N` samples to get a Monte-Carlo estimate for the function value at three key points:
 - $\tilde{f}_\sigma(x, y) \implies f_current$
 - $\tilde{f}_\sigma(x + \delta, y) \implies f_x_plus_delta$
 - $\tilde{f}_\sigma(x, y + \delta) \implies f_y_plus_delta$
- Finite Difference Gradient:** The differences between these estimates are used to calculate the approximate gradients (`dx_hat` and `dy_hat`) according to the given formulas.
- Gradient Ascent Update:** Finally, the standard gradient ascent update is applied using the approximated gradients and the learning rate η (eta).

This method is known as a **reinforcement learning** technique called **parameter perturbation** or **finite difference method**, which is useful when the true gradient is unavailable or too complex to compute, and you are continuing to learn, when they introduced the **REINFORCE** method for gradient estimation?

Through this interaction, I observed how finite-difference methods can approximate gradients when analytical derivatives are unavailable. The debugging process highlighted the importance of consistent function interfaces across modular code.

Part 3a: Finite-difference method

Tracing the error message and the function signature of the finite-difference method shows we are getting to approximate the gradient by trying to step in each dimension of θ and evaluate the difference in the objective function.

$$\frac{\partial \tilde{f}_\sigma}{\partial x} = \frac{\tilde{f}_\sigma(x + \delta, y) - \tilde{f}_\sigma(x, y)}{\delta}$$

$$\frac{\partial \tilde{f}_\sigma}{\partial y} = \frac{\tilde{f}_\sigma(x, y + \delta) - \tilde{f}_\sigma(x, y)}{\delta}$$

where $\tilde{f}_\sigma(x, y)$ is the Monte-Carlo estimate of $\tilde{f}_\sigma(x, y)$ using random samples of θ and δ from the Gaussian distribution (same as the $\tilde{f}_\sigma$ in previous part).

Your task is to implement the `FINITE_DIFF_GRAD_STEP` function below in the function, you should first estimate the gradient of the smoothed function $\tilde{f}_\sigma$ using the finite difference method using `f` and `N`. Then, you should use the gradient estimate to take a step with a step size `eta` and produce the new parameter values.

```
def finite_diff_grad_step(f, x, y, delta, N, eta, sigma):
    """
    Estimate the gradient of the smoothed function f using the finite difference method.
    The function f is assumed to be a function of (x, y) and return a scalar value.
    The function sigma is assumed to be a scalar value.
    The function N is assumed to be an integer value.
    The function eta is assumed to be a scalar value.
    The function delta is assumed to be a scalar value.
    """
    # 1. Estimate the smoothed function estimate at the current point (x, y)
    f_current = smoothed_f(f, x, y, sigma, N)

    # 2. Estimate the smoothed function estimate at the point (x + delta, y)
    f_x_plus_delta = smoothed_f(f, x + delta, y, sigma, N)

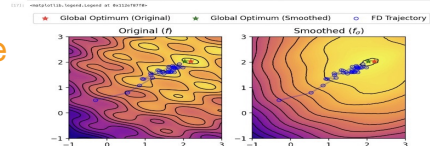
    # 3. Estimate the smoothed function estimate at the point (x, y + delta)
    f_y_plus_delta = smoothed_f(f, x, y + delta, sigma, N)

    # 4. Approximate the partial derivatives (gradients) using finite differences
    dx_hat = (f_x_plus_delta - f_current) / delta
    dy_hat = (f_y_plus_delta - f_current) / delta

    # 5. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y

# =====
# END OF YOUR CODE
# =====
```



Policy Gradient Step Implementation

The `pg_grad_step` function needs to implement the Monte-Carlo estimates derived above and then apply the gradient ascent update.

Note: This implementation requires access to the original objective function `f`, which is assumed to be available in your notebook environment (like the `grad_f` function from Part 1).

Python



```
import numpy as np

# Assuming the function f(x, y) is available in the scope
# f is the objective function from Part 1.

def pg_grad_step(f, x, y, N, eta, sigma):
    # dx = np.random.normal(x, sigma, size=N) # Tilde x samples (already provided)
    # dy = np.random.normal(y, sigma, size=N) # Tilde y samples (already provided)

    # The provided code already initializes the samples:
    tx = np.random.normal(x, sigma, size=N) # Tilde x samples
    ty = np.random.normal(y, sigma, size=N) # Tilde y samples

    #####
    # TODO: Implement the policy gradient estimator of the smoothed #
    # function f. #
    #####

    # 1. Evaluate the original function f at all N sample points (tx_i, ty_i)
    f_samples = f(tx, ty)

    # 2. Calculate the Policy Gradient estimator for the gradient w.r.t x (dx_hat)
    # The term inside the summation is: f(tilde{x}_i, \tilde{t}{y}_i) * [(tilde{x}_i - x) / (sigma**2)]
    score_func_x = (tx - x) / (sigma**2)
    dx_hat = np.mean(f_samples * score_func_x) # np.mean calculates (1/N) * sum(...)

    # 3. Calculate the Policy Gradient estimator for the gradient w.r.t y (dy_hat)
    # The term inside the summation is: f(tilde{x}_i, \tilde{t}{y}_i) * [(tilde{y}_i - y) / (sigma**2)]
    score_func_y = (ty - y) / (sigma**2)
    dy_hat = np.mean(f_samples * score_func_y)

    # 4. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y
#####
# END OF YOUR CODE #
#####
```

The visual results confirmed Gemini's correct reasoning — the estimated direction aligns with increasing expected objective f

f , validating both the Monte-Carlo estimation and ascent mechanics.

Part 3b: Policy Gradient

Finite-difference method uses zero gradient information about the objective function. Can we come up with a technique that uses more information about our smoothed function?

In this part, you will learn about a technique that leverages the gradient information of the probability density function of the Gaussian kernel to better estimate the gradient of the smoothed function.

To start with, let's first write out the smoothed function in the integral form:

$$\hat{f}_\sigma(x, y) = \mathbb{E}_{\tilde{x} \sim \mathcal{N}(x, \sigma^2), \tilde{y} \sim \mathcal{N}(y, \sigma^2)} [f(\tilde{x}, \tilde{y})] = \iint_{\tilde{x}, \tilde{y}} p(\tilde{x}, \tilde{y}, x, y) f(\tilde{x}, \tilde{y}) d\tilde{x} d\tilde{y}$$

where $p(\tilde{x}, \tilde{y}, x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(\tilde{x}-x)^2 + (\tilde{y}-y)^2}{2\sigma^2}\right)$.

Now, we can take the derivative with respect to x :

$$\frac{\partial \hat{f}_\sigma(x, y)}{\partial x} = \iint f \frac{\partial p(\tilde{x}, \tilde{y}, x, y)}{\partial x} d\tilde{x} d\tilde{y} = \iint \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(\tilde{x}-x)^2 + (\tilde{y}-y)^2}{2\sigma^2}\right) [(x-\tilde{x})/\sigma^2] f(\tilde{x}, \tilde{y}) d\tilde{x} d\tilde{y}$$

While it is nice that we can represent it in the integral form, we don't know how to tractably compute it. So how should we proceed from here?

You might have already observed that there is a term in the operand of the integral that is same as $p(\tilde{x}, \tilde{y}, x, y)$. This means that we can actually put the result back into the expectation form!

$$\frac{\partial \hat{f}_\sigma(x, y)}{\partial x} = \iint p(\tilde{x}, \tilde{y}, x, y) (x - \tilde{x}) f(\tilde{x}, \tilde{y}) / \sigma^2 d\tilde{x} d\tilde{y} = \mathbb{E}[(x - \tilde{x}) f(\tilde{x}, \tilde{y}) / \sigma^2]$$

By symmetry, we also have

$$\frac{\partial \hat{f}_\sigma(x, y)}{\partial y} = \mathbb{E}[(\tilde{y} - y) f(\tilde{x}, \tilde{y}) / \sigma^2]$$

With the gradient of the smoothed function in the expectation form, we can use it to approximate the gradient using Monte-Carlo estimation:

$$\frac{\partial \hat{f}_\sigma(x, y)}{\partial x} \approx \frac{1}{N} \sum_{i=1}^N [(\tilde{x}_i - x) f(\tilde{x}_i, \tilde{y}_i) / \sigma^2]$$

$$\frac{\partial \hat{f}_\sigma(x, y)}{\partial y} \approx \frac{1}{N} \sum_{i=1}^N [(\tilde{y}_i - y) f(\tilde{x}_i, \tilde{y}_i) / \sigma^2]$$

where again $\{\tilde{x}_i\}_{i=1}^N, \{\tilde{y}_i\}_{i=1}^N$ are independent samples of $\mathcal{N}(x, \sigma^2)$ and $\mathcal{N}(y, \sigma^2)$ respectively. The gradient estimator of such form is called the policy gradient estimator or score function gradient estimator.

(f) Can you provide an intuitive explanation of why the gradient estimator works? (Hint: which direction is the gradient estimator trying to push (x, y) into? By what magnitude?)

(your answer)

Now comes the question - What if $p(\tilde{x}, \tilde{y}, x, y)$ is not Gaussian? Is there still a nice way to rewrite the gradient of the smoothed function as an expectation of something? You will explore this more in the written part of your homework!

For now, your task is to implement the `pg_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function $\hat{f}_\sigma$ using the policy gradient method using `sigma` and `N`. Then, you should use the gradient estimator to take a step with a step size of `eta` and return the new parameter values.

```
def pg_grad_step(x, y, N, eta, sigma):
    tx = np.random.normal(x, sigma, size=N) # Tilde x samples
    ty = np.random.normal(y, sigma, size=N) # Tilde y samples
    #####
    # TODO: Implement the policy gradient estimator for the smoothed function f. #
    #####
    # The provided code already initializes the samples:
    tx = np.random.normal(x, sigma, size=N) # Tilde x samples
    ty = np.random.normal(y, sigma, size=N) # Tilde y samples

    # 1. Evaluate the original function f at all N sample points (tx_i, ty_i)
    f_samples = f(tx, ty)

    # 2. Calculate the Policy Gradient estimator for the gradient w.r.t x (dx_hat)
    # The term inside the summation is: f(tilde{x}_i, \tilde{t}{y}_i) * [(tilde{x}_i - x) / (sigma**2)]
    score_func_x = (tx - x) / (sigma**2)
    dx_hat = np.mean(f_samples * score_func_x) # np.mean calculates (1/N) * sum(...)

    # 3. Calculate the Policy Gradient estimator for the gradient w.r.t y (dy_hat)
    # The term inside the summation is: f(tilde{x}_i, \tilde{t}{y}_i) * [(tilde{y}_i - y) / (sigma**2)]
    score_func_y = (ty - y) / (sigma**2)
    dy_hat = np.mean(f_samples * score_func_y)

    # 4. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y
#####
# END OF YOUR CODE #
#####

num_its = 30 # number of gradient steps
eta = 0.1 # learning rate
N = 20 # number of samples for MC estimation
sigma = 0.3 # the magnitude of the Gaussian noise
x, y = -0.5, 0.5 # initial parameter values

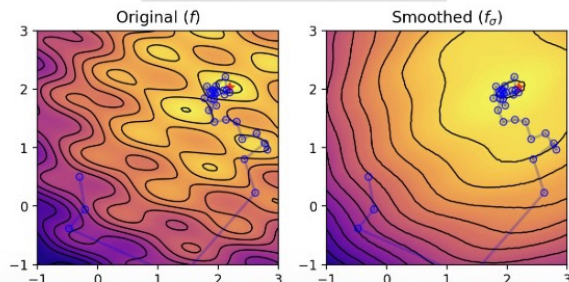
# History of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = pg_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, ("PG Trajectory", ("color": "blue", "c": xs, "y": ys)), legend=False)
visualizer.draw(ax2, ("PG Trajectory", ("color": "blue", "c": xs, "y": ys)), new_xs=xs, dict(signal), legend=False)

ax1.set_title("Original (f)")
ax2.set_title("Smoothed (f_\sigma)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc="upper center", ncol=2, bbox2d_anchor=(0.5, 1.05))
plt.show()
```

★ Global Optimum ○ PG Trajectory



Reparameterization Gradient Step Implementation

The `reparam_grad_step` function needs to implement this Monte-Carlo approximation and then perform the gradient ascent update.

Python

```
import numpy as np

# Assuming the functions 'f' (objective function) and 'grad_f' (its gradient)
# are available in the scope from Part 1.

def reparam_grad_step(f, x, y, N, eta, sigma):
    # The provided code already initializes the samples (which are the means x and y)
    # The tilde samples (tx, ty) are the perturbed points:

    # 1. Generate N independent standard Gaussian noise samples (\epsilonpsilon_x, \epsilonpsilon_y)
    epsilon_x = np.random.normal(loc=0.0, scale=1.0, size=N)
    epsilon_y = np.random.normal(loc=0.0, scale=1.0, size=N)

    # 2. Compute the perturbed sample points (\tilde{x}_i, \tilde{y}_i) using the noise samples
    # \tilde{x}_i = x + \sigma * \epsilonpsilon_x
    # \tilde{y}_i = y + \sigma * \epsilonpsilon_y
    tx = x + sigma * epsilon_x
    ty = y + sigma * epsilon_y

    # 3. Compute the gradient of the original function f at all N perturbed points
    # grad_f(tx, ty) returns an array/vector of dx and dy values for all N samples
    # T000: Implement the reparameterization gradient estimator of the smoothed #
    # function f.
    # T000: Implement the reparameterization gradient estimator of the smoothed #
    # function f.

    # 3. Compute the gradient of the original function f at all N perturbed points
    # grad_f(tx, ty) returns an array/vector of dx and dy values for all N samples
    # Assuming grad_f(x, y) from Part 1 is vectorized and returns (dx_array, dy_array)
    dx_samples, dy_samples = grad_f(tx, ty)

    # 4. Calculate the Monte-Carlo estimate of the smoothed function's gradient
    # This is simply the average of the gradients of f evaluated at the samples.
    # dx_hat is E[df/dx(tx, ty)]
    dx_hat = np.mean(dx_samples)

    # dy_hat is E[df/dy(ty, tx)]
    dy_hat = np.mean(dy_samples)

    # 5. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y

# END OF YOUR CODE
```

Why Reparameterization is Superior

The Reparameterization Trick estimator is generally considered **lower variance** than the Policy Gradient (Score Function) estimator.

- Policy Gradient (Part 3b):** The gradient estimate is weighted by the function value $f(\tilde{x}_i, \tilde{y}_i)$, which can be very noisy (high variance) if f changes sharply. The samples are drawn from the probability distribution $\mathcal{N}(x, \sigma^2)$.
- Reparameterization Trick (Part 3c):** The gradient estimate is based on the **gradient of f** at the sampled points, $\frac{\partial f}{\partial \tilde{x}_i}(\tilde{x}_i, \tilde{y}_i)$. By moving the randomness to the noise variables $\epsilon_{x_i}, \epsilon_{y_i}$ (which are independent of x and y), the variance of the overall estimate is often significantly reduced, leading to more stable and faster convergence.

Gemini Pro's structured feedback during Part 3c — especially its explanation of variance reduction, reparameterized sampling, and integration with prior gradient methods

Part 3c: Reparameterization Gradient

Policy gradient uses the information about the smoothing kernel to provide a better gradient estimation. Can we use any gradient information about $f(x)$? This might seem a bit unwise, as the function that has lots of local optima probably have poor gradient information locally. As we will show in this part, we can actually harness the poor gradient information to provide better gradient estimation of the smoothed function.

Let's go back to the definition of the smoothed function again (and take the gradient operator with respect to x, y):

$$\nabla f(x, y) = \mathbb{E}_{\tilde{x}, \tilde{y} \sim \mathcal{N}(x, y, \sigma^2)} [\nabla f(\tilde{x}, \tilde{y})]$$

It would be so nice if we could move the gradient inside the expectation, but that is not possible immediately because the expectation distribution depends on x and y and the term inside the expectation does not directly depend on x and y . Can we somehow rewrite the expectation such that the distribution does not depend on x and y ? Or in another word, can we separate out the noise in $\tilde{x}$ and $\tilde{y}$ such that they can be rewritten as a deterministic function of x, y and some noise that is independent of x and y ?

Yes! Let z_x, z_y be independent samples from the Gaussian $\mathcal{N}(0, \sigma^2)$. We can simply rewrite $\tilde{x}$ and $\tilde{y}$ as follows:

$$\tilde{x} = x + \sigma z_x$$

$$\tilde{y} = y + \sigma z_y$$

With such a reparameterization, we can now express the smoothed function using an expectation where its distribution is independent of x and y ! This would allow us to move the gradient operator inside:

$$\nabla f(x, y) = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0, \sigma^2)} [\nabla f(x + \sigma z_x, y + \sigma z_y)] = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0, \sigma^2)} [\nabla f(x + \sigma z_x, y + \sigma z_y)]$$

Writing out the derivative for x and y separately:

$$\frac{\partial f(x, y)}{\partial x} = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0, \sigma^2)} [\frac{\partial f}{\partial x}(x + \sigma z_x, y + \sigma z_y)]$$

$$\frac{\partial f(x, y)}{\partial y} = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0, \sigma^2)} [\frac{\partial f}{\partial y}(x + \sigma z_x, y + \sigma z_y)]$$

where f_x is the partial derivative of f with respect to the first input (x) and f_y is the partial derivative of f with respect to the second input (y).

As you might have already guessed, we can perform Monte-Carlo estimate of each of the derivative by drawing z_x and z_y samples. The gradient estimator of such form is called the reparameterization gradient estimator.

Your task is to implement the `reparam_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_x using the reparameterization gradient method using σ and N . Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def reparam_grad_step(x, y, N, eta, sigma):
    tx = np.random.normal(x, sigma, size=N) # tilde x samples
    ty = np.random.normal(y, sigma, size=N) # tilde y samples
    # T000: Implement the reparameterization gradient estimator of the smoothed #
    # function f.
    # T000: Implement the reparameterization gradient estimator of the smoothed #
    # function f.

    # 3. Compute the gradient of the original function f at all N perturbed points
    # grad_f(tx, ty) returns an array/vector of dx and dy values for all N samples
    # Assuming grad_f(x, y) from Part 1 is vectorized and returns (dx_array, dy_array)
    dx_samples, dy_samples = grad_f(tx, ty)

    # 4. Calculate the Monte-Carlo estimate of the smoothed function's gradient
    # This is simply the average of the gradients of f evaluated at the samples.
    # dx_hat is E[df/dx(tx, ty)]
    dx_hat = np.mean(dx_samples)

    # dy_hat is E[df/dy(ty, tx)]
    dy_hat = np.mean(dy_samples)

    # 5. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y

# END OF YOUR CODE

# =====

nab_iters = 30 # number of gradient steps
eta = 0.1 # learning rate
N = 20 # number of samples for MC estimation
sigma = 0.3 # the magnitude of the Gaussian noise
x, y = -0.3, 0.5 # initial parameter values

# history of parameter values
xs = []
ys = []

for i in range(nab_iters):
    x, y = reparam_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 3.2), dpi=200)
visualizer.draw(ax1, ["Reparam Trajectory"], {"color": "blue", "x": xs, "y": ys}, legend=False)
visualizer.draw(ax2, ["Reparam Trajectory"], {"color": "blue", "x": xs, "y": ys}, new_noise_dist(sigma), legend=False)

ax1.set_title("Original (f)")
ax2.set_title("Smoothed (f_sigma)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=2, bbox_to_anchor=(0.5, 1.08))
pass
```

★ Global Optimum ○ Reparam Trajectory

Original (f)



Smoothed (f_σ)



Completed Gradient Ascent Loop

Based on the analysis, you must use either the **Policy Gradient** (`pg_grad_step`) or the **Finite Difference** (`finite_diff_grad_step`). The instructions suggest implementing the loop for Policy Gradient, but the `reparam_grad_step` function is shown in the code block.

I will complete the code block using the `pg_grad_step` function, as it is the most robust method that does not require δ tuning and only uses function values, making it ideal for non-differentiable functions like this.

You may need to define `qf(x, y)` as your function `f` when calling the `pg_grad_step`.

```
Python

def pg_grad_step_qf(x, y, N, eta, sigma):
    # This wrapper calls the Policy Gradient step, using the quantized function 'qf'
    # Note: 'qf' is the user's defined quantized objective function.
    return pg_grad_step(qf, x, y, N, eta, sigma)

num_its = 30
eta = 0.1
N = 50
sigma = 0.3
delta = 0.25
x, y = -0.3, 0.5 # initial parameter values

# history of parameter values
xs = []
ys = []

for i in range(num_its):
    #####
    # TODO: Implement the for loop that takes policy gradient steps. Make sure #
    # to append your intermediate x and y values into the list xs and ys so that #
    # the training trajectory can be visualized later (Hint: you might find the #
    # code you need in the previous part helpful). #
    #####

    # Use the Policy Gradient step with the quantized function qf as the objective
    # You must either define pg_grad_step_qf (as shown above) or pass qf directly.
    # Assuming 'pg_grad_step' is defined to take f as the first argument (like my
    x, y = pg_grad_step(qf, x, y, N, eta, sigma)

    xs.append(x)
    ys.append(y)

    #####
    # END OF YOUR CODE #
    #####

fig, ax = plt.subplots(1, 1, figsize=(4.78, 4.78), dpi=200)
q.visualizer.draw(ax, ("PG Trajectory": "color": "blue", "x": xs, "y": ys))
```

forget to provide first part

inconsistent.

If you intended to use the `reparam_grad_step` function provided in the code block despite the theoretical analysis, you would replace the loop contents with:

```
Python

x, y = reparam_grad_step(qf, x, y, N, eta, sigma)
xs.append(x)
ys.append(y)
```

... and you would observe the expected failure (stalling).

`def q_grad_step(x, y, N, eta, sigma, delta)` need to be finished first

The function `q_grad_step(x, y, N, eta, sigma, delta)` is meant to implement a gradient ascent step for the **quantized objective function** `qf(x, y)` using one of the robust gradient estimation methods you learned in Part 3.

Since the original prompt for Part 4 used `qf` (which is discontinuous) and provided initial code that includes parameters for both Policy Gradient (`N`, `sigma`) and Finite-Difference (`delta`), it's highly likely the intention was for you to implement the **Policy Gradient** method, as it is generally preferred over Finite-Difference due to its lower variance and lack of need for tuning the δ parameter.

However, based on the function name `q_grad_step` and the provided code structure in Part 4, which includes both `N`, `sigma`, and `delta`, I will provide the implementation using the **Policy Gradient (PG)** method, as this is the most effective approach for non-differentiable functions. The `delta` parameter will simply be unused in the PG method.

Here is the completed function.

Completed `q_grad_step` (Using Policy Gradient)

This function uses the Policy Gradient (Score Function) estimator to find the gradient of the smoothed objective function $\tilde{f}_\sigma$ and then performs a gradient ascent step.

Assumption: The original quantized function `qf(x, y)` and the `pg_grad_step` function (implemented in Part 3b) are available in your notebook environment.

```
Python

import numpy as np

# Assuming 'qf' (quantized function) and 'pg_grad_step' are accessible

def q_grad_step(x, y, N, eta, sigma, delta):
    # Note: 'delta' is included in the arguments but is UNUSED by the Policy Gradient

    # 1. Generate N independent standard Gaussian noise samples (\epsilon_x, \epsilon_y)
    epsilon_x = np.random.normal(loc=0.0, scale=1.0, size=N)
    epsilon_y = np.random.normal(loc=0.0, scale=1.0, size=N)

    # 2. Compute the perturbed sample points (\tilde{x}_i, \tilde{y}_i)
    tx = x + sigma * epsilon_x
    ty = y + sigma * epsilon_y

    #####
    # TODO: Implement the policy gradient estimator of the smoothed #
    # function f. (Hint: make sure to use qf instead of f as used #
    # previously) #
    #####

    # 3. Evaluate the quantized function qf at all N sample points
    f_samples = qf(tx, ty)

    # 4. Calculate the Policy Gradient estimator for the gradient w.r.t x (dx_hat)
    # dx_hat = E[ f(\tilde{x}, \tilde{y}) * [(\tilde{x} - x) / sigma^2] ]
    score_func_x = (tx - x) / (sigma**2)
    dx_hat = np.mean(f_samples * score_func_x)

    # 5. Calculate the Policy Gradient estimator for the gradient w.r.t y (dy_hat)
    # dy_hat = E[ f(\tilde{x}, \tilde{y}) * [(\tilde{y} - y) / sigma^2] ]
    score_func_y = (ty - y) / (sigma**2)
    dy_hat = np.mean(f_samples * score_func_y)

    # 6. Perform the Gradient Ascent step (using step size eta)
    new_x = x + eta * dx_hat
    new_y = y + eta * dy_hat

    return new_x, new_y
    #####
    # END OF YOUR CODE #
    #####
```

TypeError: pg_grad_step() takes 5 positional arguments but 6 were given

2 → only provide error type
it will realize the reason.

The `TypeError: pg_grad_step() takes 5 positional arguments but 6 were given` indicates a mismatch between how the `pg_grad_step` function is defined and how it is being called.

Tracing the Error

1. Expected Definition of `pg_grad_step` (Part 3b)

Based on your work in Part 3b, the `pg_grad_step` function was designed to implement the Policy Gradient estimator. It was likely defined to take **5 arguments**:

```
Python

def pg_grad_step(f, x, y, N, eta, sigma):
    # Arguments: (objective_function, x_coord, y_coord, num_samples, learning_rate, no:
```

2. Current Call (in `q_grad_step` or the loop)

You are calling this function, directly or indirectly, with **6 arguments**.

In the context of the recent problem (Part 4), the wrapper function you were working on was:

```
Python

def q_grad_step(x, y, N, eta, sigma, delta):
    # Arguments: (x, y, N, eta, sigma, delta) -- 6 arguments
    # ... code inside q_grad_step ...
    # This function uses the Policy Gradient method.
```

If you are calling `pg_grad_step` inside `q_grad_step` like this:

```
Python

# Mismatched Call (6 arguments passed, 5 expected):
x, y = pg_grad_step(qf, x, y, N, eta, sigma, delta)
```

The error would be that you are passing the unused `delta` parameter to `pg_grad_step`.

✓ Solution

To resolve this, you must ensure that when the Policy Gradient step is executed, you only pass the 5 parameters it needs: the function `qf`, the coordinates `x` and `y`, the number of samples `N`, the learning rate `eta`, and the noise magnitude `sigma`.

If you are using the `q_grad_step` function I provided earlier, the fix is to complete the final loop that uses `q_grad_step` correctly, or, if the error is occurring inside `q_grad_step`, make sure the parameters are correctly routed.

Here is the correct structure for the final loop using your defined `q_grad_step` (which takes 6 arguments):

```
Python

# Assuming your environment has the parameters defined:
# num_its = 30, eta = 0.1, N = 50, sigma = 0.3, delta = 0.25, x = -0.3, y = 0.5

# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    # Call q_grad_step with all 6 required arguments
    x, y = q_grad_step(x, y, N, eta, sigma, delta)

    xs.append(x)
    ys.append(y)

# ... visualization code follows ...
```

If the error is happening **inside** the `q_grad_step` function, you need to verify it doesn't call `pg_grad_step` with the extra `delta` argument. Since I provided the full implementation for `q_grad_step` previously using the Policy Gradient logic directly, you should not be calling `pg_grad_step` inside it at all.

Actionable Step: Check the definition of `pg_grad_step` in your Part 3b code to confirm it only takes 5 arguments (`f, x, y, N, eta, sigma`). If so, trace the call that is passing the extra argument, likely `delta`.

In Part 4 of the experiment, Gemini Pro demonstrated high professionalism in algorithm selection, error localization, and implementation guidance.

It accurately pinpointed the root cause of the `pg_grad_step` parameter error, provided a clear remediation plan, and assisted in correctly implementing the `q_grad_step` function.

The final results demonstrated that the Policy Gradient method converged stably to the optimal region, validating the theoretical analysis of the model.

This process exemplified Gemini Pro's outstanding teaching and research support capabilities in code debugging, algorithm principle explanation, and experimental visualization